

Strukture podataka i algoritmi

Stek, red, heš tabele

dr Danijel Čabarkapa

Akademija strukovnih studija Šabac

Odsek za medicinske i poslovno-tehnološke studije

Sadržaj:

1. Struktura podataka: Stek
2. Struktura podataka: Red
3. Struktura podataka: Heš tabele

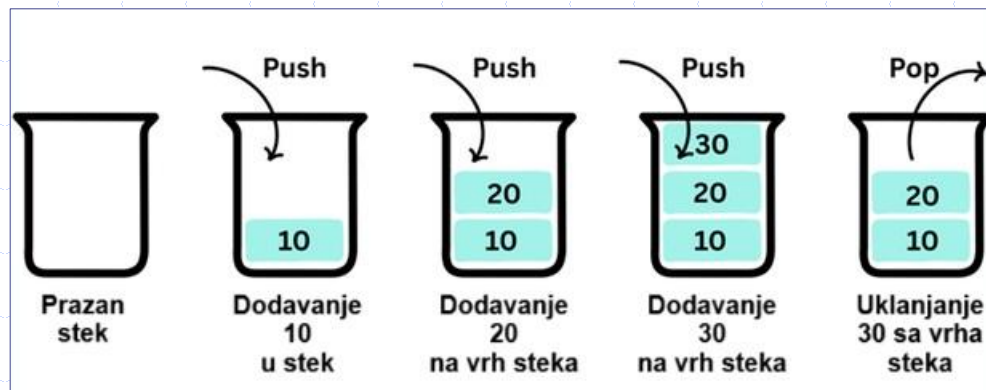
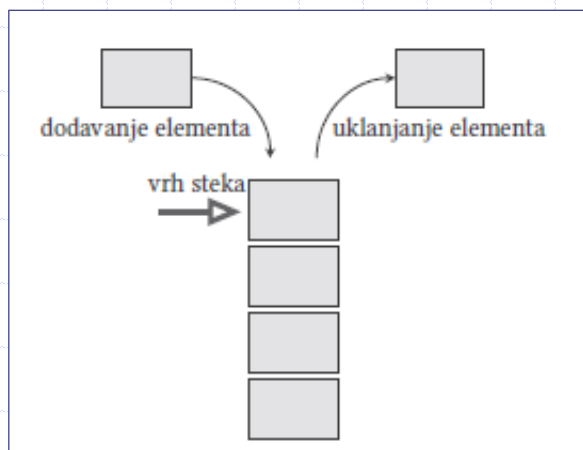
1. Struktura podataka: Stek

Stek struktura podataka

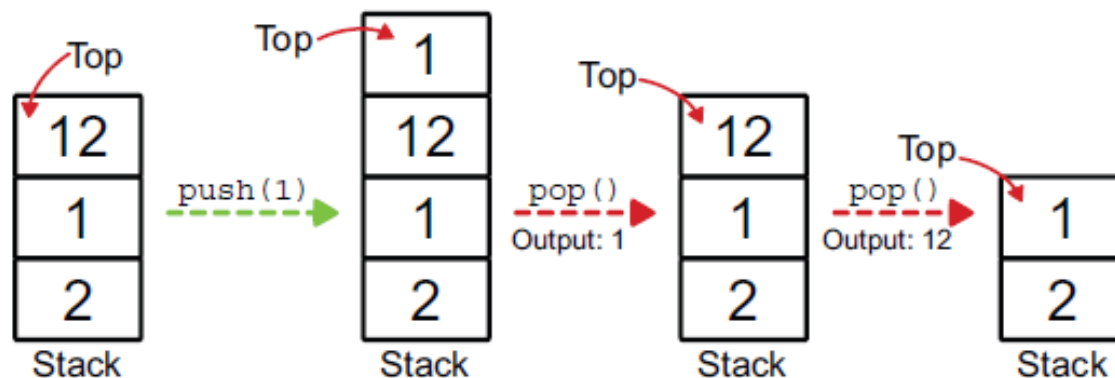
- **Stek** (*stack*) je linearna i dinamička struktura podataka koja funkcioniše po principu **LIFO** (*Last In First Out*)
- Stek je struktura podataka koja se, kao i povezana lista ili niz, sastoji od jednog niza podataka, ali kod koje **elemente možemo dodavati i uklanjati na samo jednom kraju**
- LIFO princip: poslednji dodati element u stek prvi je koji se može iz njega uzeti, tj. poslednji element koji je ušao, prvi je koji ide van
- Ovu struktura podataka karakterišu dve specifične operacije – **push** za dodavanje elemenata i **pop** za njihovo uklanjanje
- Stek se može implementirati na više načina: kao niz fiksne veličine, dinamički niz ili jednostruko/dvostruko povezana lista
- Stek teorijski ne podržava operacije arbitrarnog pretraživanja ili umetanja novih elemenata bilo gde u listi

Stek struktura podataka

- Kod steka su elementi niza poređani „jedan na drugi“, pri čemu se sva dodavanja i uklanjanja elemenata steka izvršavaju samo na vrhu steka (*top of the stack*)
- Stek koristi dve specifične operacije – **push** za **dodavanje** elemenata i **pop** za njihovo **uklanjanje**
- Poredak elemenata po kojem se oni uklanjaju sa vrha steka je obrnut onom po kojem su oni dodati na vrh steka

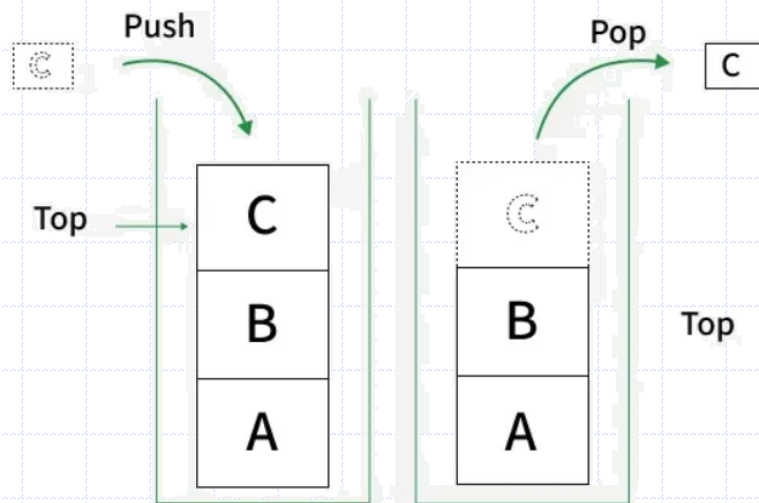


Stek struktura podataka



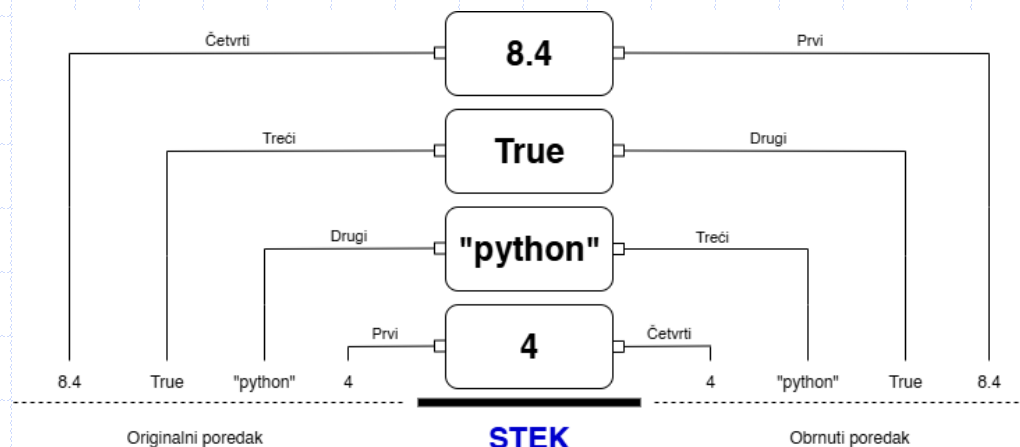
- U skladu sa ograničenjem LIFO principa, kod steka se održava redosled dodavanja novih elemenata i potrebno je pratiti poslednji element koji je dodat na stek, a koji se naziva vrh steka (**Top**)
- U praksi, formira se niz elemenata steka prema redosledu njihovog dodavanja, a vrh steka će biti strana niza gde dodajemo i uklanjamo elemente (bez obzira na to kako je niz sačuvan)

Osnovne operacije steka



- Za stek se definišu sledeće operacije:
 - `push(c)`: dodavanje novog elementa na vrh steka
 - `pop()`: brisanje elementa sa vrha steka
 - `size()`: vraća broj elemenata steka
 - `isEmpty()`: proverava da li je stek prazan
 - `top()/peek()`: vraća vrednost elementa na vrhu, bez uklanjanja istog

Osnovne operacije steka



Stek operacija	Sadržaj steka	Povratna vrednost
s.is_empty()	[]	True
s.push(4)	[4]	
s.push('python')	[4, 'python']	
s.peek()	[4, 'python']	'python'
s.push(True)	[4, 'python', True]	
s.size()	[4, 'python', True]	3
s.is_empty()	[4, 'python', True]	False
s.push(8.4)	[4, 'python', True, 8.4]	
s.pop()	[4, 'python', True]	8.4
s.pop()	[4, 'python']	True
s.size()	[4, 'python']	2

Osnovne operacije steka

- Python liste obezbeđuju sopstvene ugrađene (built-in) metode koje se mogu direktno koristiti za rad sa stekom
 - `append()` metodom se dodaje element na kraj liste
 - `pop()` metoda vraća vrednost elementa na vrhu liste (steka)

```
1  stack = []
2
3  # append() funkcija za unos elementa u stek
4  stack.append('a')
5  stack.append('b')
6  stack.append('c')
7
8  print('Početni stek:')
9  print(stack)
10 # pop() funkcija uklanja element po LIFO principu
11 print('\nElementi uklonjeni iz steka:')
12 print(stack.pop())
13 print(stack.pop())
14 print(stack.pop())
15
16 print('\nStek nakon uklonjenih elemenata:')
17 print(stack)
18
19 # iapisom print(stack.pop()) javlja IndexError
20 # grešku jer stek nije prazan
```

Početni stek:
['a', 'b', 'c']

Elementi uklonjeni iz steka:
c
b
a

Stek nakon uklonjenih elemenata:
[]

Stek – implementacija

- Python ne poseduje ugrađeni tip podatka tipa stek, pa se implementacija steka vrši upotrebom drugih struktura podataka
- Stek se može implementirati na sledeće načine:
 - 1) preko nizova
 - stek fiksne dužine (statički niz)
 - stek promenljive dužine (dinamički niz)
 - 2) preko povezanih listi
- Implementacija steka preko povezanih listi nije jednostavno primenljiva, ali je efikasna
- Kod implementacije steka putem statičkog niza (stek ima fiksnu veličinu), važno je proveriti da li je stek pun ili ne, jer će pokušaj unošenja elementa u pun stek generisati grešku prekoračenja (*overflow*)

Stek kao statički niz

Dodavanje elemenata u stek push()

01 Step Empty Stack

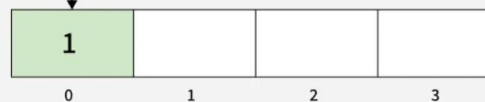
Capacity = 4

top = -1



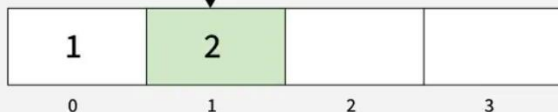
02 Step Push Element 1 Into Stack

top = 0



03 Step Push Element 2 Into Stack

top = 1



04 Step Push Element 3 Into Stack

Capacity = 4

top = 2

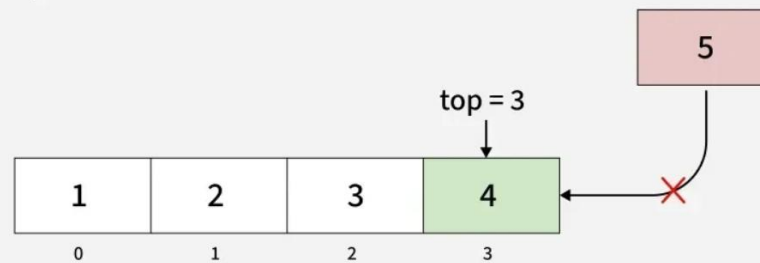


05 Step Push Element 4 Into Stack

top = 3



06 Step Push Element 5 Into Stack (Stack Overflow)



Stek kao statički niz

- **Uklanjanje elemenata iz steka pop()**, elementi se uklanjaju obrnutim redosledom od unosa, ako je stek prazan, onda nastaje uslov Underflow
 - Pre uklanjanja elementa steka proverava se da li je stek prazan
 - Ako jeste, (`top==-1`), nastao je **Underflow** i ne može se ukloniti ni jedan element iz steka
 - U suprotnom, sačuvamo vrednost `top`, smanjimo je za 1 (`top=top-1`) i vraćamo sačuvanu vrednost za `top`

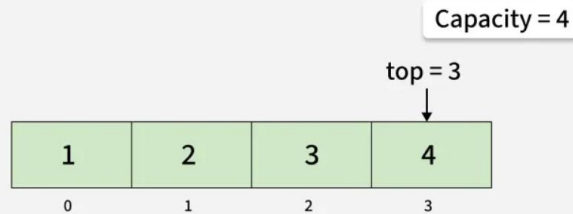
```
1  def pop(self):
2
3      if self.top == -1:
4          print("Stack Underflow")
5          return -1
6
7      value = self.arr[self.top]
8      self.top -= 1
9
10     return value
```

- Vremenska složenost za `pop()` steka je $O(1)$
- Prostorna složenost za `pop()` steka je $O(1)$

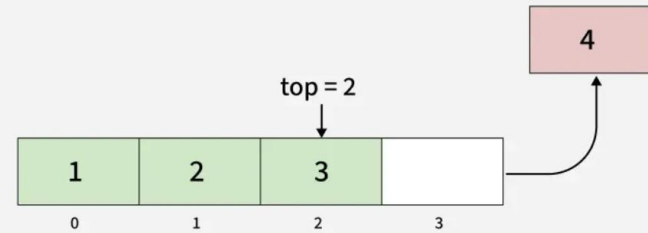
Stek kao statički niz

Uklanjanje elemenata iz steka pop()

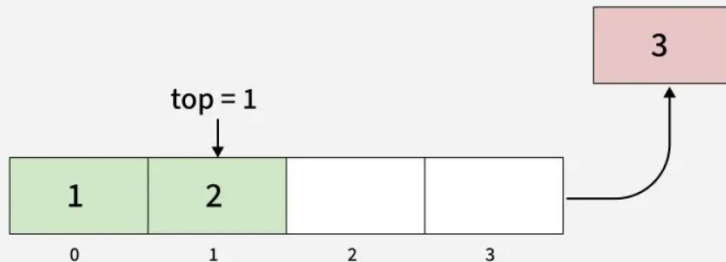
01 Step Initial Stack



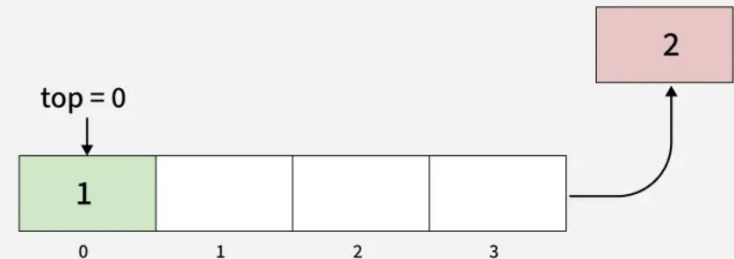
02 Step Pop Element 4 From Stack



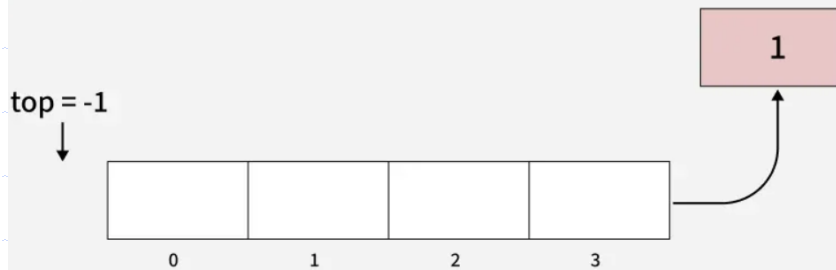
03 Step Pop Element 3 From Stack



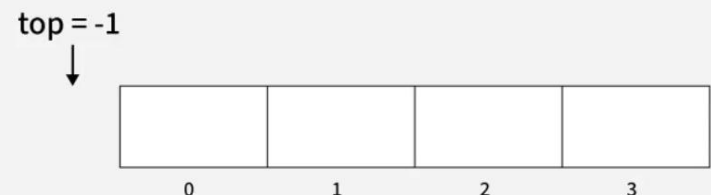
04 Step Pop Element 2 From Stack



05 Step Pop Element 1 From Stack



06 Step Empty Stack (Popping an Element will cause underflow)



Stek kao statički niz

- Operacija **peek()**, vraća element sa vrha steka, ako je stek prazan (**top == -1**), štampa se "Stek je prazan", u suprotnom se vraća indeks **top** elementa
- Operacija **isEmpty()**, vraća **True** ako je stek prazan, u suprotnom **False**, proverava se vrednost **top** elementa, ako je (**top == -1**) stek je prazan i vraća se **True**
- Operacija **isFull()**, vraća **True** ako je stek popunjen, u suprotnom **False**, proverava se vrednost **top** elementa, ako je (**top == capacity - 1**) stek je popunjen i vraća se **True**, u suprotnom **False**

```
1 def peek(self):
2
3     if self.top == -1:
4         print("Stack is Empty")
5         return -1
6
7     return self.arr[self.top]
```

```
1 def isEmpty(self):
2
3     return self.top == -1

1 def isFull(self):
2     return self.top == self.capacity - 1
```

- Vremenska složenost za **peek()**, **isEmpty()**, **isFull()** je **O(1)**
- Prostorna složenost za **peek()**, **isEmpty()**, **isFull()** je **O(1)**

Stek – primene

- **Pozivi funkcija:** Stek se koristi za praćenje poziva funkcija, omogućavajući programu da se vrati na ispravnu lokaciju nakon što je funkcija završila sa izvršavanjem
- **Rekurzija:** Stek se koristi za čuvanje lokalnih promenljivih i povratnih adresa rekurzivnih poziva funkcija, omogućavajući programu da prati trenutno stanje rekurzije
- **Evaluacija izraza:** Stek se koristi za evaluaciju izraza u postfiksnoj notaciji (Reverse Polish Notation)
- **Parsiranje sintakse:** Stek se koristi za proveru validnosti sintakse u programskim jezicima i drugim formalnim jezicima
- **Upravljanje memorijom:** Stek se koristi za dodeljivanje i upravljanje memorijom u nekim operativnim sistemima i programskim jezicima
- Koristi se za rešavanje „popularnih“ problema kao što su: Next Greater, Previous Greater, Next Smaller, Previous Smaller, Largest Area in a Histogram and Stock Span Problems

Stek – prednosti i nedostaci

- Prednosti steka:

- Jednostavnost - stek je jednostavna i lako razumljiva struktura podataka, primenljiva za širok opseg zadataka
- Efikasnost - **push** i **pop** operacije imaju const. vremensku složenost $O(1)$, pa je pristup podacima steka efikasan
- LIFO princip: efikasan i koristan u programiranju, od poziva funkcija, preko izračunavanja izraza ...
- Ograničeno korišćenje memorije: stek samo čuva elemente koji se dodaju komandom **push**, pa je memorijski efikasniji od drugih struktura

- Nedstaci steka:

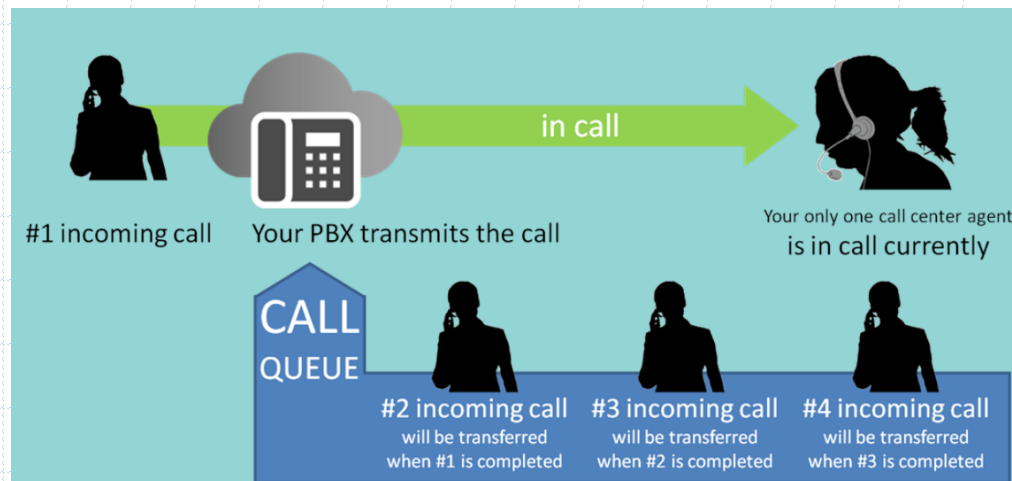
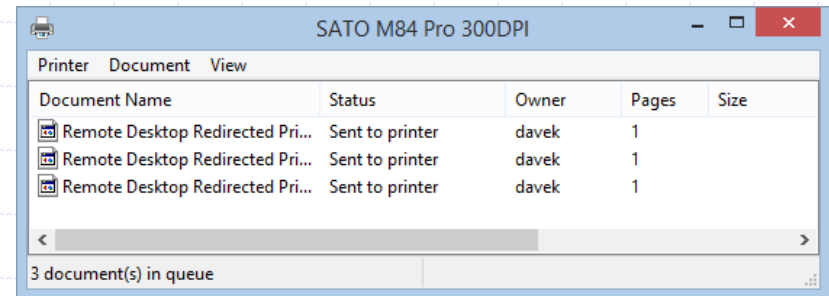
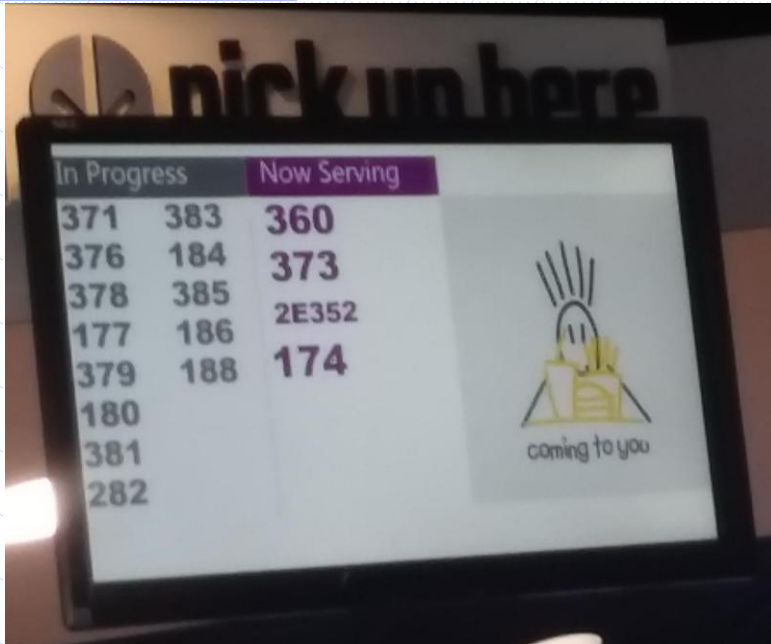
- Ograničen pristup: pristup elementima steka preko **top** člana, problem pristupa i izmene elemenata u sredini
- Nema mogućnost slučajnog pristupa elementima
- Ograničen kapacitet steka: stek ima ograničen kapacitet, pa se ne može upotrebiti za slučaj čuvanja većeg ili nepoznatog broja elemenata
- Prekoračenje (overflow): ako se unese veći broj elemenata nego što stek podržava, javlja se prekoračenje i mogući gubitak podataka

2. Struktura podataka: Red

Red

- **Red** (*queue*) ili „red za čekanje“ je linearna i dinamička struktura podataka koja funkcioniše po principu **FIFO** (*First In First Out*)
- Red je struktura podataka koja se, kao i povezana lista ili niz, sastoji od niza podataka, kod koje **elemente možemo dodavati na jednom kraju, a uklanjati ih na drugom**
- **FIFO princip**: Prvi dodati element u red prvi je koji se može iz njega ukloniti, dodavanje elemenata je moguće u bilo kom trenutku, ali je moguće ukloniti samo onaj element koji je najduže u redu, elementi reda se dodaju na kraj, a uklanjaju se sa početka reda
- Red na šalteru je primer strukture red, osoba koja prva stigne, prva će završiti posao na šalteru, dok će osobe koje su kasnije došle dolaziti na kraj reda i čekati...

Redovi - primene



Redovi - primene

PRIMENA REDOVA



CPU Scheduling

CPU zadaci su postavljeni u red i izvršavaju se jedan za drugim



Job Scheduling

Štampači štampaju višestruke dokumente koji su u formi reda čekanja



Networking

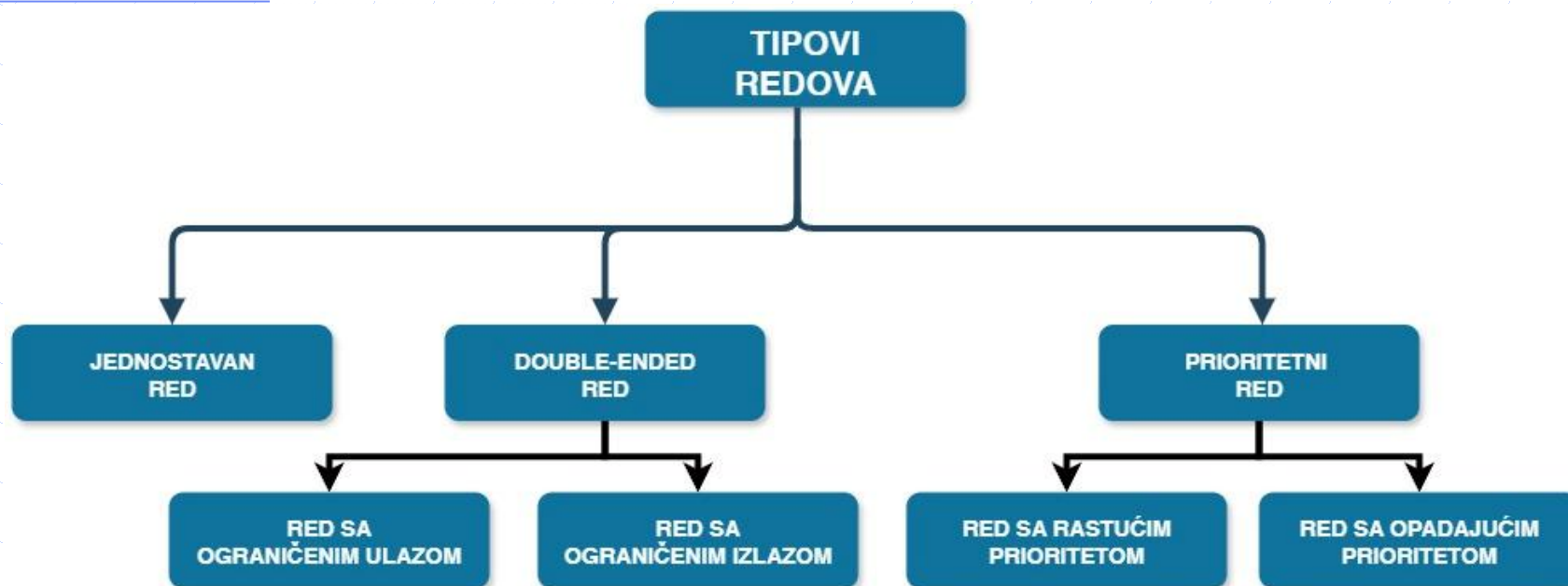
Ruteri, svičevi i mrežni uređaji obrađuju podatke koji su u formi redova



Buffer Management

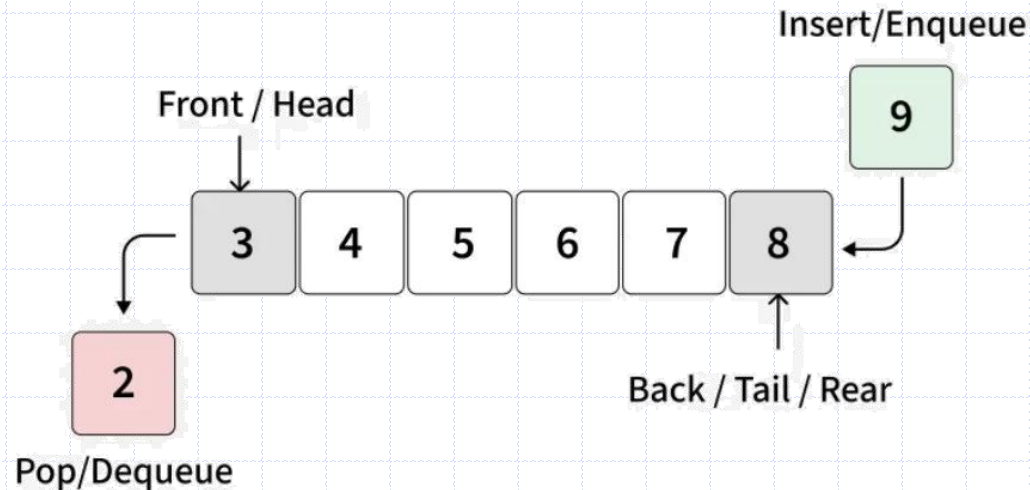
Koriste se kao buffer memorija između sporih i brzih uređaja (keyboard-CPU network devices)

Vrste redova



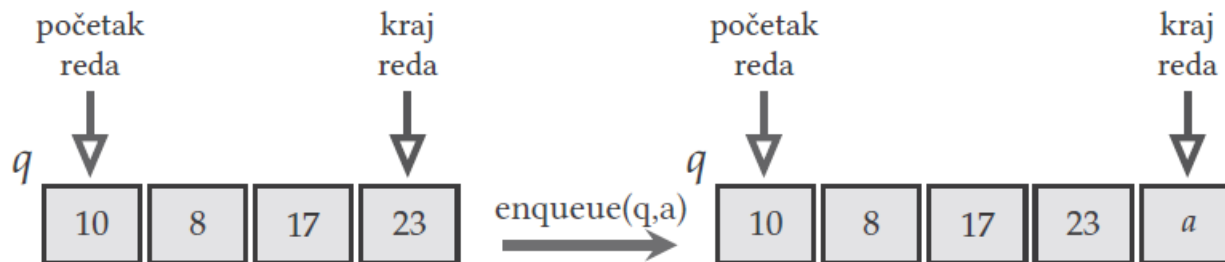
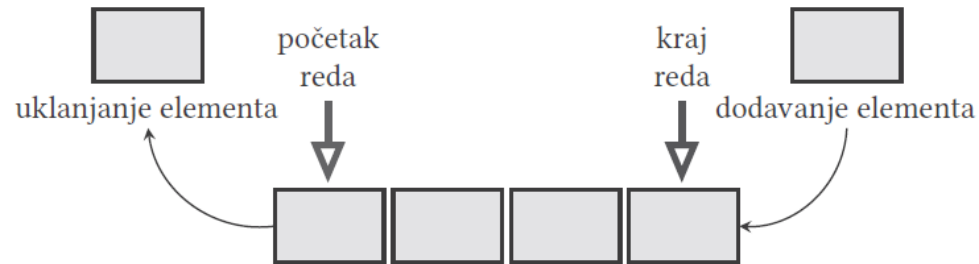
- **Jednostavan red**: FIFO princip (unos samo na kraju-rear, brisanje samo na početku-front, kreira se kroz povezane ili kružne liste)
- Red sa dva kraja ili **double-ended** (*deque*): unos ili brisanje može da se vrši na oba kraja
- **Red sa prioritetom**: svakom elementu dodeljen je prioritet, a brisanje se uvek dešava na osnovu prioriteta a ne samo pozicije elementa

Red - terminologija

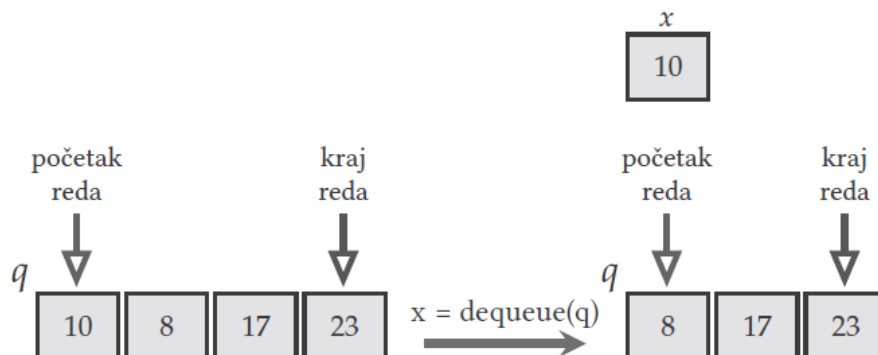


- Za redove su definišu dve ključne operacije:
 - a) dodavanje elemenata na kraj reda **Enqueue**
 - b) Uklanjanje elemenata sa početka reda **Dequeue**
- **Front/Head**: Pozicija prvog dodatog elementa u redu koji će biti procesiran i prvi koji će biti uklonjen iz reda
- **Rear/Tail/Back**: Pozicija poslednjeg unetog elementa u red
- **Size**: Predstavlja tekući broj elemenata reda
- **Capacity**: Maksimalni broj elemenata koji mogu da se čuvaju u redu

Red



Enqueue
(dodavanje elementa **a** na kraj reda **q** koji sadrži elemente 10,8,17 i 23)

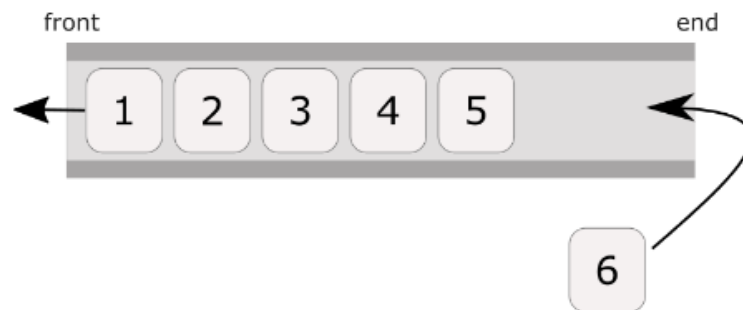


Dequeue
(uklanjanje prvog elementa **10** sa početka reda **q** koji sadrži elemente 10,8,17 i 23)

Osnovne operacije sa redovima

- Osnovne operacije za rad sa redovima su:
 - `enqueue(x, Q)`: ubacuje element `x` na kraj reda `Q`, ako je red pun dešava se prekoračenje `overflow` – vremenska složenost: $O(1)$
 - `dequeue(Q)`: uklanja i vraća element sa početka reda `Q`, ako je red prazan dešava se `underflow` uslov – vremenska složenost: $O(1)$
 - `front(Q)`: vraća element sa početka reda `Q`, ne menjajući ga – vremenska složenost: $O(1)$
 - `rear(Q)`: vraća element sa kraja reda `Q`, ne menjajući ga – vremenska složenost: $O(1)$
 - `make_null(Q)`: pretvara red `Q` u prazni red
 - `empty(Q)`: vraća `True` ako je red `Q` prazan, u suprotnom vraća `False`
- Pošto `Python` liste imaju potpunu funkcionalnost potrebnu za implementaciju redova, kreiranje reda i izvođenje operacija nad njim moguće je postići jednostavno, u par linija programskog kôda
- Drugi način je implementacija redova pomoću `klasa` i `objekata`

Osnovne operacije sa redovima



Operacija reda	Sadržaj reda	Povratna vrednost
D.enqueue(2)	[2]	-
D.enqueue(4)	[2, 4]	-
D.dequeue()	[4]	2
D.empty()	[4]	False
D.dequeue()	[]	4
D.empty()	[]	True
D.dequeue()	[]	Error: Emmpy queue
D.enqueue(7)	[7]	-
D.enqueue(5)	[7, 5]	-
D.dequeue()	[7, 5, 4]	7

Osnovne operacije sa redovima

- Red se može jednostavno implementirati pomoću niza (liste)
- Dodavanje elemenata u red preko `append()` a uklanjanje sa početka reda preko `pop(0)`

```
1 queue = []
2
3 # Enqueue metoda
4 queue.append('A')
5 queue.append('B')
6 queue.append('C')
7 print("Red: ", queue)
8
9 # Dequeue metoda
10 element = queue.pop(0)
11 print("Dequeue: ", element)
12
13 # Peek metoda
14 frontElement = queue[0]
15 print("Peek: ", frontElement)
16
17 # isEmpty metoda
18 isEmpty = not bool(queue)
19 print("Prazan red: ", isEmpty)
20
21 # Veličina reda
22 print("Veličina reda: ", len(queue))
```



```
Red: ['A', 'B', 'C']
Dequeue: A
Peek: B
Prazan red: False
Veličina reda: 2
```

Implementacija reda preko niza

- Za implementaciju reda pomoću niza fiksne veličine koristimo:
 - Niz fiksne veličine `arr[]` za čuvanje elemenata reda
 - Promenljivu `size` za praćenje trenutnog broja elemenata reda
 - Promenljivu `capacity` za čuvanje max. broja elemenata reda

```
1 class myQueue:
2     def __init__(self, capacity):
3
4         # Max. broj elemenata koji red može da čuva
5         self.capacity = capacity
6
7         # Niz za čuvanje elemenata reda
8         self.arr = [0] * capacity
9
10        # Tekući broj elemenata reda
11        self.size = 0
```

```
1 def enqueue(self, x):
2     if self.size == self.capacity:
3         print("Queue Overflow")
4         return
5
6     self.arr[self.size] = x
7     self.size += 1
8
```

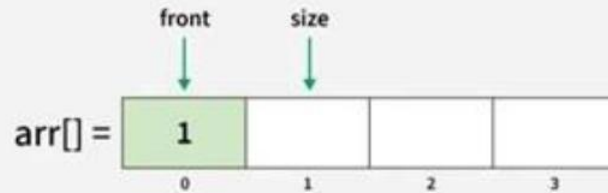
```
1 def dequeue(self):
2     if self.size == 0:
3         print("Queue Underflow")
4         return
5
6     for i in range(1, self.size):
7         self.arr[i-1] = self.arr[i]
8     self.size -= 1
```

Implementacija reda preko niza

01
Step

enqueue(1): unet element u red, index je tekući size, red nije prazan, početno: front=0, size=0, capacity=4
arr[0]=1

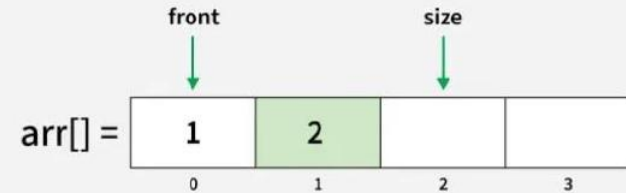
size = size + 1



02
Step

enqueue(2): unet element u red, index je tekući size, red nije prazan, front=0, size=1, capacity=4
arr[1]=2

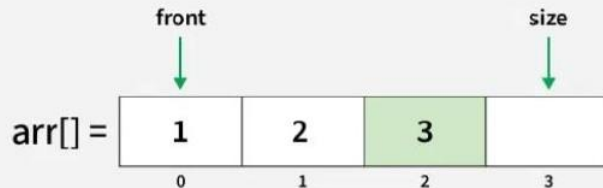
size = size + 1



03
Step

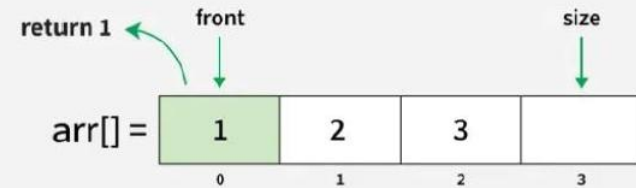
enqueue(3): unet sledeći element u red, index je tekući size, red nije prazan, front=0, size=2, capacity=4
arr[2]=3

size = size + 1



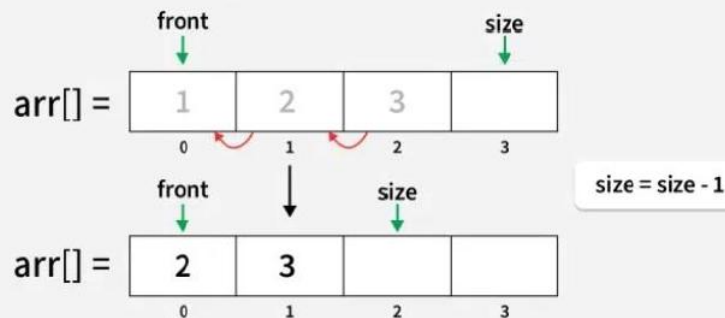
04
Step

getFront(): vraća front element reda, ako size nije 0, u suprotnom vraća -1



05
Step

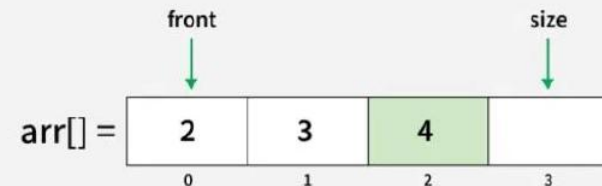
dequeue(): uklanja front element reda pomeranjem svih ostalih elemenata reda jedno mesto u levo, smanjujući size za 1



06
Step

enqueue(4): unosi se sledeći element u red, index je tekući size, red nije popunjen front=0, size=2, capacity=4
arr[2]=4

size = size + 1

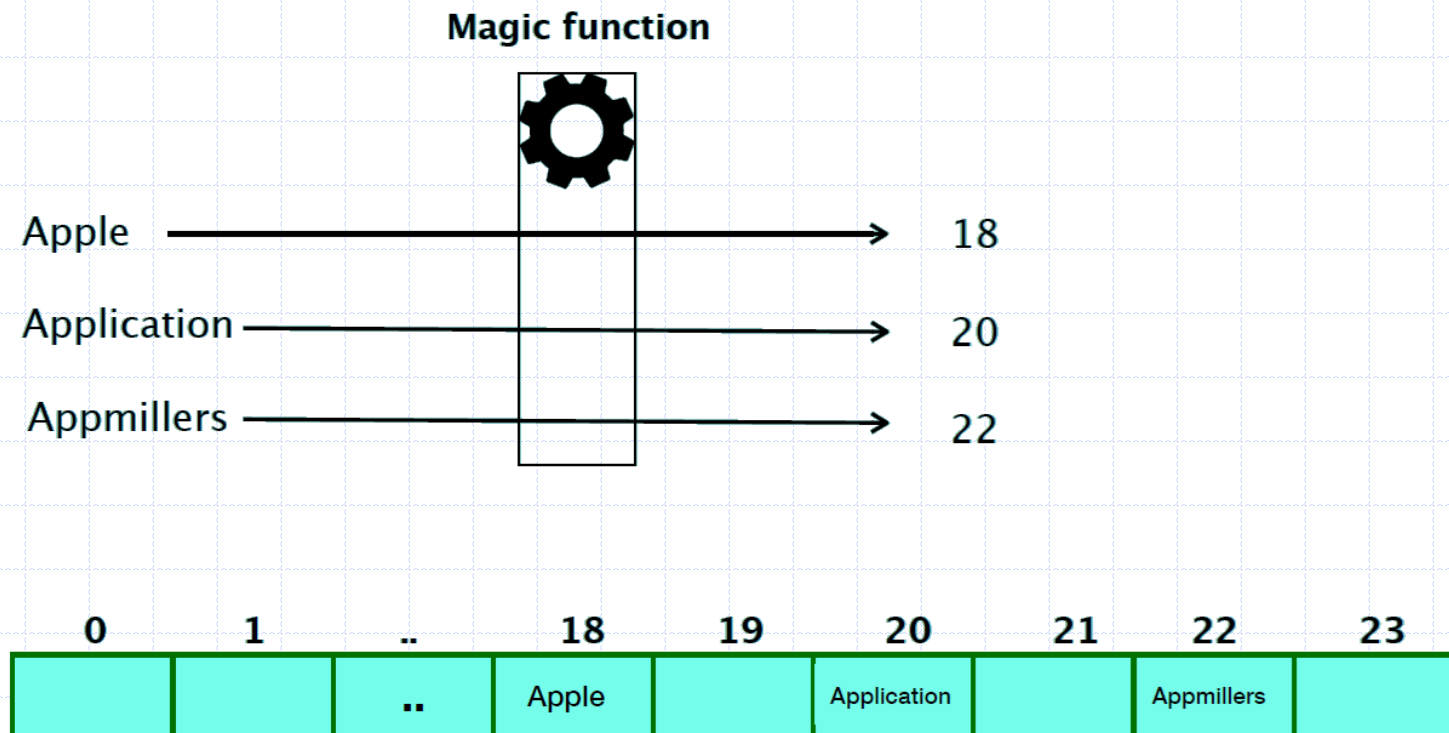


3. Struktura podataka: Heš tabele

Pojam heširanja

Pojam **heširanja**:

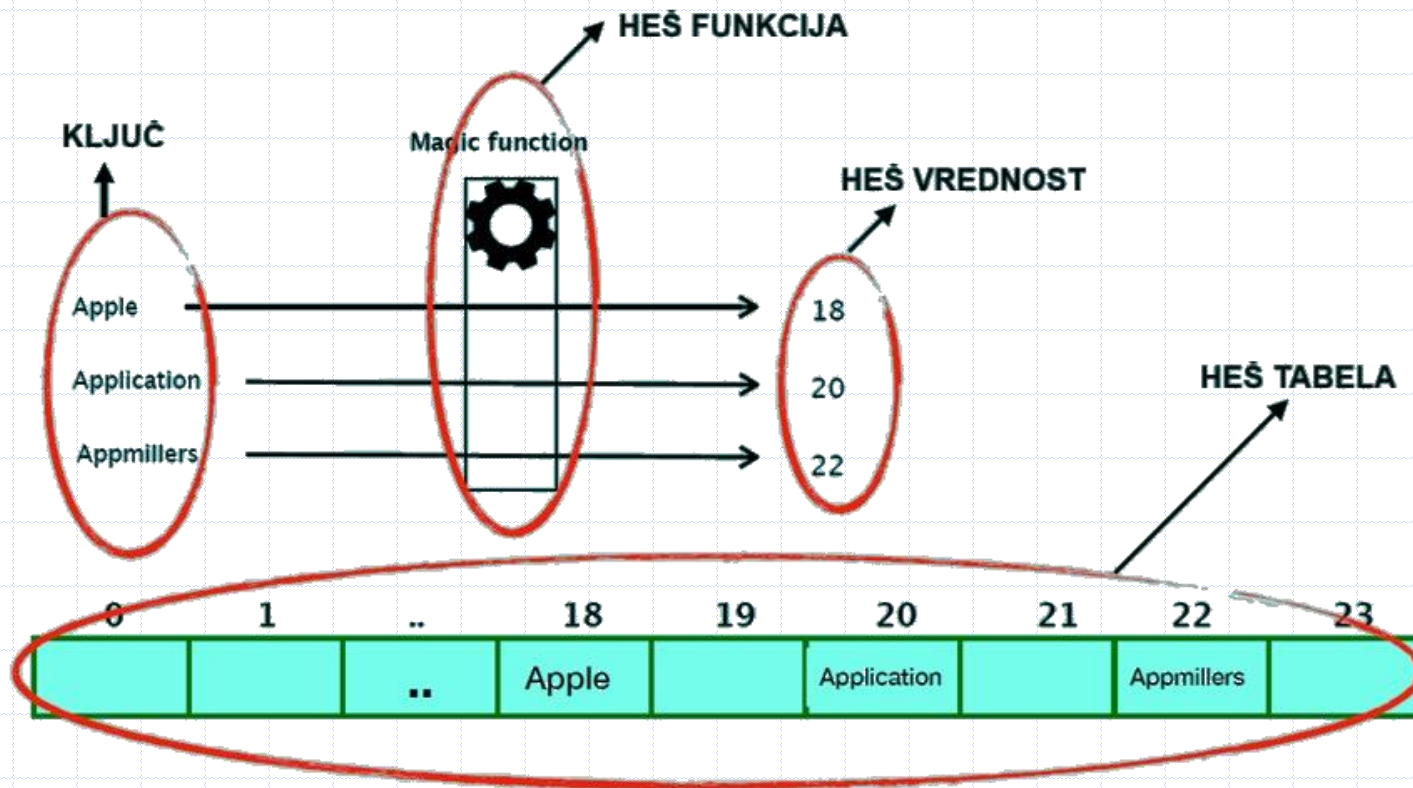
Heširanje je metoda sortiranja i indeksiranja podataka kod koje se indeksiranje velike količine podataka vrši ključevima koji se kreiraju matematičkom funkcijom



Heš tabele

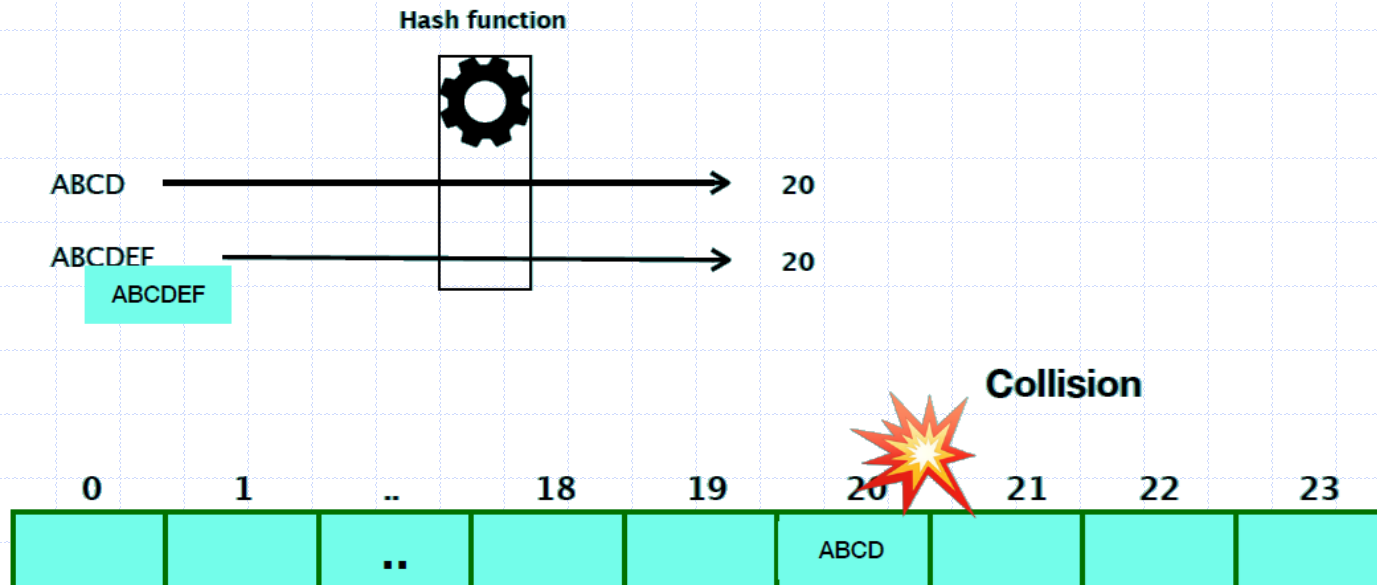
Heš tabela (hash table):

Struktura podataka koja implementira apstraktni tip podataka asocijativnog niza postupkom mapiranja parova ključ/vrednost (key/value) i koja je promenljiva (*mutable*)

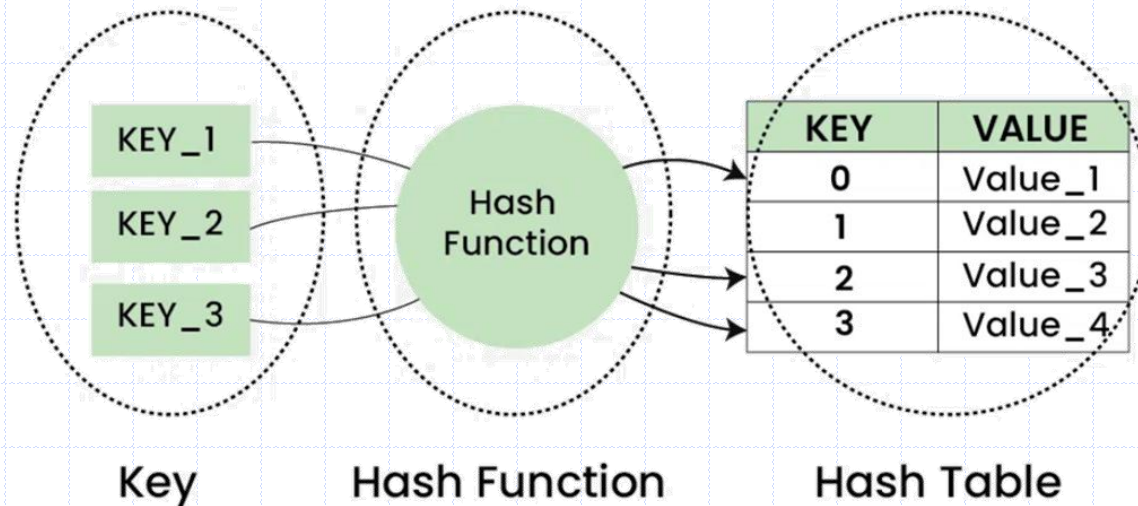


Komponente heširanja

- Ključ (key) predstavlja ulazni podatak
- Heš funkcija je funkcija za mapiranje podataka proizvoljne veličine na podatke fiksne veličine
- Heš vrednost je vrednost koju vraća heš funkcija
- Heš tabela je struktura podataka koja implementira niz kao tip podataka kroz mapiranje ključeva u vrednosti
- Kolizija (collision) nastaje kada dva različita ključa heš funkcije formiraju isti izlaz



Komponente heširanja



Tri komponente heširanja:

Ključ može biti string ili ceo broj koji je ulaz u heš funkciju, koja matematičkom funkcijom određuje indeks ili lokaciju za skladištenje elementa u strukturi podataka

Heš funkcija prihvata ulazni ključ i vraća indeks elementa u nizu koji se naziva heš tabela. Indeks predstavlja heš indeks

Heš tabela je struktura podataka koja mapira ključeve u vrednosti koristeći funkciju koja se naziva heš funkcija. Heš tabela čuva podatke u formi asocijativanog niz, gde svaka vrednost podataka ima svoj jedinstveni indeks

Heš tabele

- Heš tabela predstavljaju uopštenje standardnih nizova
- Heš tabela koristi niz čija je veličina proporcionalna broju ključeva koji su stvarno uskladišteni
- Za razliku od tabela da direktnim adresiranjem (*direct address table*) kod kojeg su ključevi numerički indeksi, heš tabele koriste indekse koji se računaju iz ključa
- Kada je broj uskladištenih ključeva mali u odnosu na ukupan broj mogućih ključeva, heš tabele predstavljaju alternativu za efektivno i direktno adresiranje niza
- Heš tabele su efikasne strukture podataka za implementaciju REČNIKA, u Python jeziku rečnik je jedna od osnovnih struktura
- Iako pretraga elemenata u heš tabeli može da traje kao i pretraga u povezanoj listi, u praksi se heširanje pokazalo kao dobro rešenje

Heš tabela

- Heš tabela se naziva i „tabela raspršivanja“
- Da bi se ostvarile podjednako dobre performanse kao kod indeksiranja niza, potrebno je u što većoj meri izbeći traženje ključa
- Osnovna ideja heširanja je da se umesto pretrage ključa njegov položaj u nizu izračunava na osnovi samog ključa, čime se izbegava nepotrebno pretraživanje velikog broja ključeva
- Heš tabela je struktura podataka koja može efikasno raditi sa brojem aktivnih (upotrebljenih) ključeva koji je puno manji od broja svih mogućih ključeva
- Ključ heš tabele ne mora isključivo biti numeričkog tipa i to se postiže upotrebom **heš funkcije** (*hash function*)

Heš tabele

- Iako skladištenje niza traje $O(1)$ vremena, pretraživanje u njemu traje najmanje $O(\log n)$ vremena
- Za veliki skup podataka to može biti sporo, i učiniti niz neefikasnim
- Potrebna je struktura koja može da skladišti podatke i pretražuje ih u konstantnom vremenu, tj. u $O(1)$
- Uvođenjem heš tabela, moguće je lako čuvati i preuzimati podatke u konstantnom vremenu $O(1)$
- **Prednosti heširanja** u strukturama podataka:
 - Heširanje je efikasan način za struktura ključ-vrednost
 - Heširanje omogućava brz pristup elementima sa konstantnom vremenskom složenošću
 - Operacije umetanja, brisanja i pretraživanja su veoma efikasne
 - Heširanje zahteva manje memorije jer dodeljuje fiksni prostor za skladištenje elemenata
 - Heširanje dobro funkcioniše sa velikim skupovima podataka, održavajući konstantno vreme pristupa
 - Heširanje je neophodno za bezbedno skladištenje podataka i verifikaciju integriteta

Heš funkcija

Lista = [11, 12, 13, 14, 15]

$$H(x) = [x \% 10]$$

	11%10	12%10	13%10	14%10	15%10
	↓	↓	↓	↓	↓
0	1	2	3	4	5
Hash Table	11	12	13	14	15

```
def mod(number, cellNumber):  
    return number % cellNumber
```

`mod(400, 24)` → 16

`mod(700, 24)` → 4

0	1	..	4	5	..	16	..	23
		..	700		..	400	..	

S obzirom da heš tabela u pozadini upotrebljava niz za smeštanje ključeva i vrednosti, ključ se mora transformirati u numeričku vrednost koja odgovara nekom indeksu tog niza

Primer heš funkcije:

Funkcija **modulo** (deljenje sa ostatkom), koja vraća ostatak pri celobrojnomo deljenju

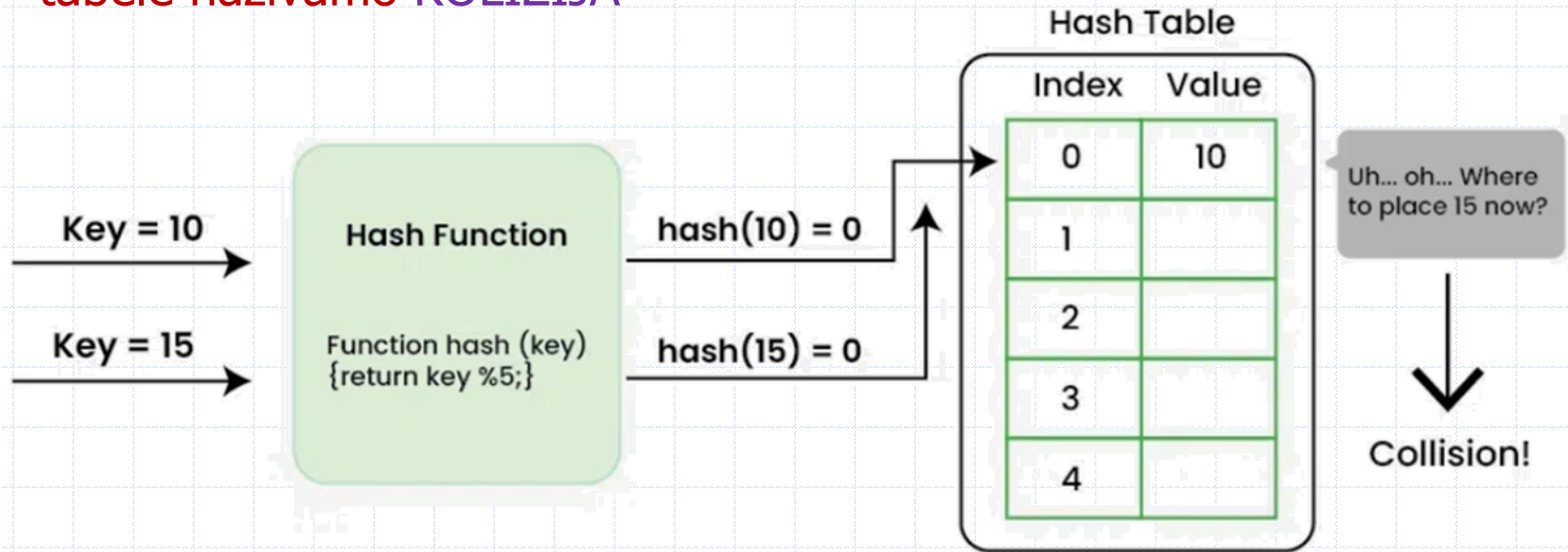
$$13\%10=3$$

Dobra heš funkcija

- **Ravnomerna distribucija ključeva** heš tabele poželjna je karakteristika heš funkcije
- Jedan pojam koji je više od teorijskog značaja je nezavisna ravnomerna heš funkcija (*independent uniform hash function*)
- To je "idealna" heš funkcija kod koje je za svaki ključ k (iz skupa svih mogućih ključeva) svaka vrednost $h(k)$ iz domena $\{0, 1, \dots, m-1\}$ jednako verovatna ili $h(k)$ je element koji je slučajno i nezavisno od drugih elemenata odabran iz skupa elemenata $\{0, 1, \dots, m-1\}$
- **Dobra heš funkcija** aproksimativno zadovoljava pretpostavku jednostavnog, uniformnog heširanja: svaki ključ ima jednaku verovatnoću da se smesti u jednu od m ćelija, bez obzira gde su drugi ključevi heširani
- U praksi je obično nemoguće proveriti ovaj uslov, jer najčešće nije poznata verovatnoća distribucije ključeva

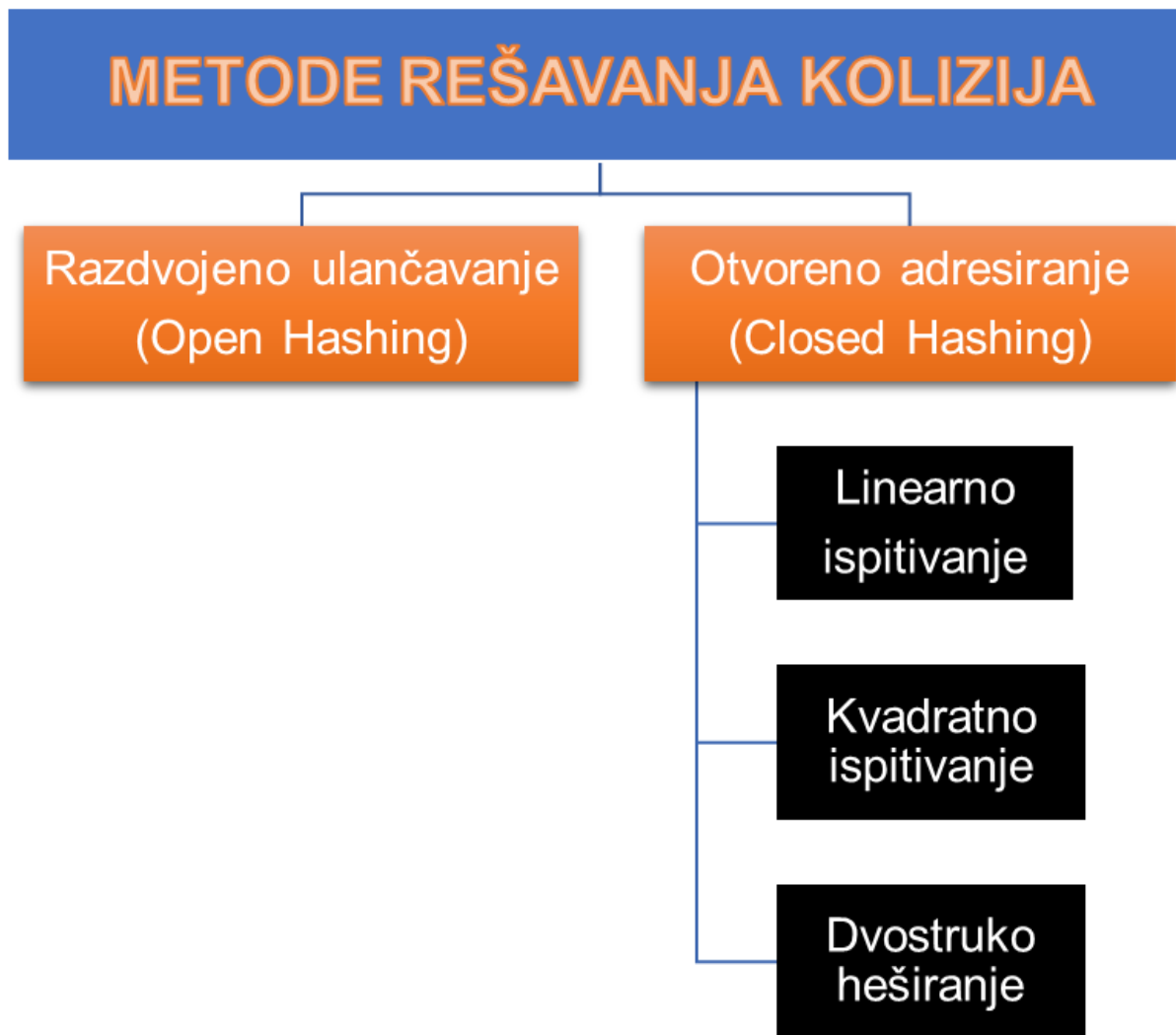
Kolizija heš tabela

- Pojavu kada se dva ili više ključeva moraju smestiti na isti indeks heš tabele nazivamo **KOLIZIJA**



- Kada je broj upotrebljenih ključeva veći od kapaciteta niza u koji se oni smeštaju, neki će indeksi morati da sadrže više ključeva pa postoji mogućnost da dva ključa imaju istu vrednost. Problem kada se novoumetnuti ključ mapira na već zauzet ključ rešava se neom od metoda za rukovanje kolizijama
- Ne postoji najbolja tehnika da se otkloni konflikt izazvan kolizijom

Metode rešavanja kolizija kod heširanja



Rešavanje kolizija ulančavanjem

- **Metoda ulančavanja** ili odvojenog niza je (*separate chaining*) vrši se heš funkcijom $h(x) = h_1(x)$
- Ulančavanje je tehnika rešavanja kolizija kod koje se svi ključevi koje treba smestiti na isti indeks niza heš tabele sačuvaju u povezanoj listi, zajedno s adresom pripadajuće vrednosti
- Ako treba pronaći zadati ključ, pretražuje se ta povezana lista, obično linearnim pretraživanjem
- Pristup ključu kod ovakve strukture podataka sastoji se u tome da se prvo odredi indeks na kojem se on nalazi upotrebom heš funkcije, a nakon toga unutar liste na tom se indeksu linearnim pretraživanjem pronađe se traženi ključ
- Za analizu performansi heš tabele koristi se vrednost **faktor opterećenja α** (load factor)
- Za heš tabelu veličine **m** i broj ključeva **n** faktor opterećenja iznosi **$\alpha = n/m$** i ta vrednost može biti veća, manja ili jednaka 1

Rešavanje kolizija ulančavanjem

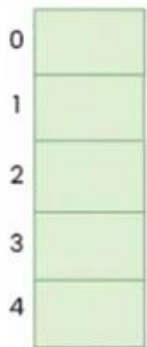
Primer:

Data je heš funkcija, uneti elemente u heš tabelu metodom ulančavanja

Heš funkcija = $\text{key} \% 5$

Elementi = 12, 15, 22, 25, 37

Slot



0	
1	
2	
3	
4	

KORAK 1

prazna heš tabela sa opsegom vrednosti od 0 do 4 u skladu sa datom heš funkcijom



Slot



0	
1	
2	12
3	
4	

Korak 2

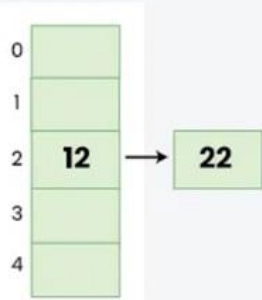
Prvi ključ koji treba uneti je 12, koji se mapira u slot 2

Rešavanje kolizija ulančavanjem

Heš funkcija = $\text{key} \% 5$

Elementi = 12, 15, 22, 25, 37

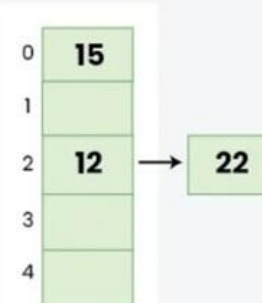
Slot



KORAK 3

Sledeći ključ je 22, mapira se u slot 2 jer je $22 \% 5 = 2$, u slotu 2 je već upisan ključ 12, formira se povezana lista za slot 2

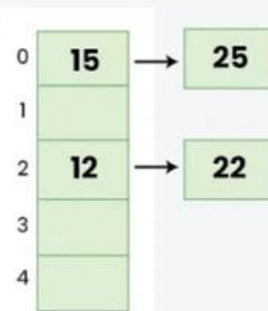
Slot



KORAK 4

Sledeći ključ je 15 i on se mapira u slot 0 jer je $15 \% 5 = 0$

Slot



KORAK 5

Sledeći ključ je 25, mapira se u slot 0 jer je $25 \% 5 = 0$, u slotu 0 je već upisan ključ 15, za slot 0 se formira povezana lista

Heširanje - primene

Password verification

Login Required

User:

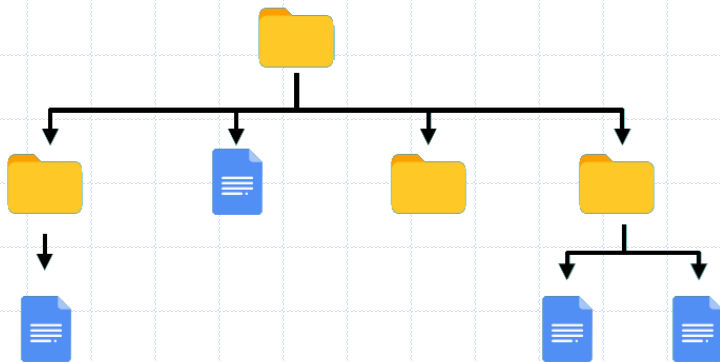
Password:

Login Cancel

Google Servers



File system : File path is mapped to physical location on disk



0	
1	/Documents/ Files/hashing.txt
2	
3	

